

NEIGHBORHOOD-ENRICHED MULTI-SCALE SHORTEST-PATH KERNEL NETWORKS

Anonymous authors

Paper under double-blind review

ABSTRACT

Graph Kernel Neural Networks (GKNNs) such as Hierarchical Shortest-Path Graph Kernel Networks (HSP-GKN) unify non-parametric graph kernel expressivity with end-to-end differentiable optimization. However, existing shortest-path kernel networks suffer from **path node context blindness**: because they map paths directly from static node attributes X , nodes possessing identical raw attributes $a(u) = a(v)$ contribute identically to step visit matrices M_l regardless of surrounding graph topology. Prior non-linear or continuous state space extensions attempt to resolve this but break degree-1 scale homogeneity, causing kernel scale corruption and RKHS ill-conditioning on unnormalized continuous attribute datasets, severe parameter inflation, and training slowdowns. In this paper, we propose **Neighborhood-Enriched Multi-Scale Shortest-Path Kernel Networks (NEM-SPKN)**, an architecture that resolves path context blindness while rigorously preserving exact linear attribute scale and computational parsimony. NEM-SPKN incorporates multi-hop neighborhood feature diffusion $T^k X$ via a purely linear, provably degree-1 homogeneous enrichment $Z = [X \mid X_{\text{nbr}}Q]$ capped at D_{NBR} channels. Crucially, because Z is strictly linear, the feature mapping factorizes into parameter-free precomputed path matrices $\text{vec}(Z^\top M_l) = [\text{vec}(X^\top M_l) \mid (I_{l+1} \otimes Q^\top) \text{vec}(X_{\text{nbr}}^\top M_l)]$, allowing NEM-SPKN to maintain HSP-GKN training speed while operating under a strictly controlled parameter budget ($1.34\times$ average parameter ratio). Evaluated across all 12 benchmark TUDatasets, NEM-SPKN achieves 10 wins out of 12 against reproduced HSP-GKN (mean accuracy gain $+0.84$ points), yielding substantial improvements on MUTAG (96.3%), NCI1 (83.4%), IMDB-B (79.2%), and REDDIT-B (93.4%). Comprehensive ablations validate the necessity of mathematical scale preservation, shallow diffusion ($K = 2$), channel capacity capping ($D_{\text{NBR}} = 32$), symmetric normalization, and explicit channel concatenation.

1 INTRODUCTION

Graph classification is a fundamental problem in machine learning on structured data, with applications spanning bioinformatics, drug discovery, chemoinformatics, and social network analysis Kriege et al. (2019); Nikolentzos et al. (2019); Borgwardt et al. (2020); Morris et al. (2020). Two dominant paradigms have emerged for graph-level representation learning. Traditional graph kernels, such as shortest-path and Weisfeiler-Lehman subtree kernels, offer non-parametric guarantees and well-defined Reproducing Kernel Hilbert Spaces (RKHS) Borgwardt et al. (2020); Morris et al. (2018), but rely on static, hand-engineered features and decoupled SVM classifiers, precluding end-to-end optimization. Conversely, spatial GNNs Wu et al. (2019b); Zhou et al. (2018); Xu et al. (2018) learn task-specific features via message passing, yet are upper-bounded in expressivity by the 1-Weisfeiler-Leman (1-WL) test Xu et al. (2018); Morris et al. (2018) and suffer from over-smoothing and over-squashing Alon & Yahav (2020); Giovanni et al. (2023); Chen et al. (2019).

To unify the theoretical advantages of graph kernels with the end-to-end optimization of deep learning, Graph Kernel Neural Networks (GKNNs) have attracted substantial research interest Lei et al. (2017); Cosmo et al. (2021); Du et al. (2019); Wang et al. (2025). In particular, the recently proposed Hierarchical Shortest-Path Graph Kernel Network (HSP-GKN) Wang et al. (2025) bridges this gap by mapping graphs into hierarchical path feature vectors organized by shortest-path lengths. HSP-GKN precomputes discrete path-step visit count matrices $M_l \in \mathbb{R}^{n \times (l+1)}$ for

each shortest-path length $l \in \{1, \dots, \delta(G)\}$, where entry $[M_l]_{i,j}$ records how many times node i appears at step j along shortest paths of length l . By evaluating the hierarchical feature mapping $\phi_l(G) = \text{vec}(X^\top M_l)$, where $X \in \mathbb{R}^{n \times d}$ represents the node attribute matrix, HSP-GKN eliminates the prohibitive $O(|V|^2|V'|^2)$ pairwise path comparison overhead of classical GraphHopper and shortest-path kernels Hermansson et al. (2015); Nikolentzos et al. (2017a), enabling linear scaling in dataset size and fast end-to-end training.

Despite its computational efficiency, HSP-GKN suffers from a fundamental theoretical limitation: **Path Node Context Blindness**. Because HSP-GKN computes path feature representations directly from the static node attribute matrix X , any two nodes u and v possessing identical raw attributes $a(u) = a(v)$ contribute identically to the path visit matrix M_l . Consequently, the model treats path steps in complete isolation, ignoring the rich k -hop local neighborhood topology surrounding u and v along each path traversal Draief et al. (2018); Nikolentzos et al. (2017b). Recent attempts to remedy this context blindness by introducing non-linear feature lifts, neural activations, or continuous state space transformations fail in practice: non-linear activations break the degree-1 scale homogeneity $Z(\alpha X) = \alpha Z(X)$ inherent to linear graph kernels, inducing kernel scale corruption, RKHS feature ill-conditioning, and float32 overflow risk under unnormalized continuous node attributes (Appendix A.6), alongside severe parameter inflation (often blowing up parameter budgets by $20\times$) and computational slowdowns.

To overcome path node context blindness while rigorously preserving exact linear attribute scale, computational efficiency, and parameter parsimony, we propose **Neighborhood-Enriched Multi-Scale Shortest-Path Kernel Networks (NEM-SPKN)**. NEM-SPKN introduces a purely linear, provably degree-1 scale-homogeneous neighborhood attribute diffusion operator $X_{\text{nbr}} = [TX \mid T^2X \mid \dots \mid T^KX] \in \mathbb{R}^{n \times (dK)}$, where $T = D^{-1/2}AD^{-1/2}$ is the symmetric normalized adjacency matrix. To prevent parameter explosion on high-dimensional attribute graphs, NEM-SPKN appends a compact learned neighborhood projection $X_{\text{nbr}}Q$ capped by a global channel capacity bound D_{NBR} , yielding an enriched node representation $Z = [X \mid X_{\text{nbr}}Q] \in \mathbb{R}^{n \times (d+m)}$, where $m = \min(dK, D_{\text{NBR}})$.

Crucially, because the enrichment Z is strictly linear in X , the hierarchical path feature mapping factorizes cleanly through the precomputed path visit matrices: $\phi_l(G) = \text{vec}(Z^\top M_l) = [\text{vec}(X^\top M_l) \mid Q^\top \text{vec}(X_{\text{nbr}}^\top M_l)]$. As a result, the parameter-free neighborhood path features $\text{vec}(X_{\text{nbr}}^\top M_l)$ can be precomputed alongside M_l in a single preprocessing pass. During training, NEM-SPKN applies lightweight linear transformations in feature space without per-node scatter operations or non-linear recomputations, preserving HSP-GKN’s fast training speed while enriching every path step with multi-scale local structural context.

The main contributions of this work are summarized as follows:

- **Identification of Path Node Context Blindness:** We analyze the structural expressivity limits of shortest-path kernel networks, proving that static node attribute mappings in HSP-GKN suffer from path node context blindness when nodes share identical attributes across different neighborhood topologies.
- **Mathematical Scale Preservation & Factorized Architecture:** We propose NEM-SPKN, which constructs a purely linear, degree-1 scale-homogeneous neighborhood enrichment $Z = [X \mid X_{\text{nbr}}Q]$ with a channel capacity bottleneck D_{NBR} . We prove that this linear enrichment exacts degree-1 scale homogeneity $Z(\alpha X) = \alpha Z(X)$ for all $\alpha \in \mathbb{R}$ and factorizes through path count matrices, avoiding numerical instability on continuous attribute datasets and enabling HSP-GKN-level training speeds.
- **Comprehensive Benchmark & Ablation Evaluation:** We conduct extensive empirical evaluations across all 12 benchmark datasets from the standardized TUDatasets collection Morris et al. (2020). NEM-SPKN outperforms the reproduced HSP-GKN baseline on 10 out of 12 datasets under strictly matched parameter budgets. Systematic ablation studies confirm the necessity of mathematical linearity, shallow diffusion depth ($K = 2$), channel capacity capping ($D_{\text{NBR}} = 32$), symmetric degree normalization, and explicit channel concatenation over residual fusion.

2 RELATED WORK

Our work lies at the intersection of classical graph kernels, expressive spatial graph neural networks, and learnable graph kernel neural networks. Below, we review relevant literature across these three domains.

2.1 SHORTEST-PATH GRAPH KERNELS AND FAST GRAPH MAPPINGS

Graph kernels measure the topological and semantic similarity between graphs by mapping combinatorial graph structures into Hilbert feature spaces Kriege et al. (2019); Nikolentzos et al. (2019); Borgwardt et al. (2020). Among structural kernels, shortest-path (SP) kernels Borgwardt et al. (2020); Hermansson et al. (2015) compare pairs of shortest paths between nodes. Classical variants include the Weisfeiler-Lehman Shortest-Path kernel (WL-SP), GraphHopper (GH) kernel, and CORE-SP Nikolentzos et al. (2018), which employs k -core decomposition to capture multi-scale subgraph hierarchies. To compare continuous node distributions, Wasserstein Weisfeiler-Lehman (WWL) Togninalli et al. (2019) applies optimal transport distances over node embeddings, while matching node embeddings Nikolentzos et al. (2017b) and ordered-neighborhood kernels (KONG) Draief et al. (2018) align local vertex contexts.

Despite their theoretical appeal, classical shortest-path kernels are (i) non-learnable, unable to adapt to downstream objectives, and (ii) computationally quadratic $O(|V|^2|V'|^2)$ due to pairwise path comparisons. Kernel Graph CNNs Nikolentzos et al. (2017a) and KerGNNs Feng et al. (2022) mitigate this via trainable graph filters or local patch embeddings, and HSP-GKN Wang et al. (2025) introduced the hierarchical mapping $\phi_l(G) = \text{vec}(X^\top M_l)$, aggregating equal-length shortest paths via precomputed visit matrices M_l for linear scaling. However, because HSP-GKN relies on the static attribute matrix X , it evaluates path steps in isolation, incurring path node context blindness whenever distinct structural nodes share identical attributes Draief et al. (2018); Nikolentzos et al. (2017b). NEM-SPKN solves this by diffusing multi-hop neighborhood attributes X_{nbr} along path steps while retaining exact discrete visit indexing via M_l .

2.2 EXPRESSIVE GRAPH NEURAL NETWORKS AND SPATIAL MESSAGE-PASSING

Spatial Message-Passing Neural Networks (MPNNs) Gilmer et al. (2017); Hamilton et al. (2017) recursively aggregate features from immediate local neighborhoods. Key architectures include GraphSAGE Hamilton et al. (2017), Graph Isomorphism Network (GIN) Xu et al. (2018), and hierarchical pooling networks such as DiffPool Ying et al. (2018) and Deep Graph Convolutional Neural Network (DGCNN) Zhang et al. (2018). Standard spatial MPNNs are theoretical bounded in discriminative power by the 1-dimensional Weisfeiler-Lehman (1-WL) graph isomorphism test Xu et al. (2018); Morris et al. (2018).

Beyond formal expressivity limits, deep MPNNs suffer from propagation bottlenecks on long-diameter graphs—over-smoothing Chen et al. (2019) and over-squashing Alon & Yahav (2020); Giovanni et al. (2023); Topping et al. (2021), where multi-hop information is compressed into fixed-size vectors. Dynamic rewiring (DRew Gutteridge et al. (2023)), identity-aware GNNs You et al. (2021), and substructure-counting networks (GSN Bouritsas et al. (2020)) alleviate these bottlenecks but incur high overhead or break local message-passing. In contrast, NEM-SPKN bypasses iterative message passing by explicitly encoding global shortest-path step distributions in M_l while enriching each step with scale-preserved multi-hop neighborhood context Morris et al. (2020).

2.3 GRAPH KERNEL NEURAL NETWORKS AND DIFFERENTIABLE KERNELS

To combine the expressive power of neural networks with the structural regularization of graph kernels, several hybrid architectures have been proposed. Early work by Lei et al. (2017) derived deep recurrent modules from sequence and graph kernels, while Graph Neural Tangent Kernels (GNTK) Du et al. (2019) characterized the infinite-width limits of GNNs trained via gradient descent. Graph Kernel Neural Networks (GKNN) Cosmo et al. (2021) replace spatial convolutions with structural kernel filters. More recently, Path Neural Networks (PathNN) Michel et al. (2023) aggregate paths emanating from nodes to exceed 1-WL expressivity, DHAKR Qian et al. (2025) introduces feature-channel attention over hierarchical substructure invariants, GNN-LoFI Biccato

et al. (2023) evaluates localized feature distributions via histogram intersection kernels, and Graph Parsing Networks (GPN) Song et al. (2024) adaptively infer hierarchical pooling structures.

A central challenge in differentiable graph kernels is maintaining numerical stability and scale preservation during end-to-end optimization Keriven et al. (2020). In HSP-GKN Wang et al. (2025), similarity is an inner product between hierarchical features $\phi_l(G)$ and learnable prototypes W_l . Prior attempts to enrich its static X via non-linear feature lifts break degree-1 scale homogeneity $Z(\alpha X) \neq \alpha Z(X)$, causing kernel scale corruption and RKHS ill-conditioning under unnormalized continuous attributes (Appendix A.6) and parameter inflation. NEM-SPKN instead uses a provably degree-1 homogeneous, purely linear enrichment $Z = [X \mid X_{\text{nbr}}Q]$ with a channel bottleneck D_{NBR} ; by factorizing through precomputed path matrices it preserves attribute scale exactly, prevents parameter bloat Morris et al. (2020), and matches HSP-GKN’s training speed.

3 METHODOLOGY

In this section, we present the mathematical formulation and structural architecture of Neighborhood-Enriched Multi-Scale Shortest-Path Kernel Networks (NEM-SPKN). First, we establish formal graph notation and define path node context blindness. Second, we introduce scale-preserved multi-hop neighborhood diffusion. Third, we formulate linear scale-preserving neighborhood enrichment and prove its degree-1 scale homogeneity. Finally, we detail the factorized path feature mapping and end-to-end optimization framework.

3.1 PRELIMINARIES AND PROBLEM FORMULATION

Let $G = (V, E, a)$ be an undirected attributed graph, where $V = \{v_1, v_2, \dots, v_n\}$ is the vertex set ($n = |V|$), $E \subseteq V \times V$ is the edge set, and $a : V \rightarrow \Sigma$ assigns node attributes from domain Σ . We represent node attributes as a matrix $X \in \mathbb{R}^{n \times d}$, where the i -th row corresponds to attribute vector $a(v_i)^\top \in \mathbb{R}^d$. The unweighted adjacency matrix is $A \in \mathbb{R}^{n \times n}$. A path $\pi = (v_1, v_2, \dots, v_k)$ of discrete length $|\pi| = k - 1$ connects v_1 to v_k via E . The diameter $\delta(G)$ is the maximum shortest-path length between any node pair in G .

Static HSP Path Visit Matrices: For each shortest-path length $l \in \{1, 2, \dots, \delta(G)\}$, HSP-GKN Wang et al. (2025) uses Dijkstra’s algorithm to compute a step visit matrix $M_l \in \mathbb{R}^{n \times (l+1)}$. Entry $[M_l]_{i,j}$ records how many times node v_i appears at index $j \in \{0, 1, \dots, l\}$ along all shortest paths of discrete length l in G . The static hierarchical path feature mapping for length l is defined as:

$$\phi_l(G) = \text{vec}(X^\top M_l) \in \mathbb{R}^{d(l+1)}, \quad (1)$$

where $\text{vec}(\cdot)$ denotes column-vectorization stacking $d \times (l+1)$ matrix entries into a single vector.

Theorem 3.1 (Path Node Context Blindness). *Let $u, v \in V$ be two distinct nodes in G with identical raw node attributes $a(u) = a(v) = x \in \mathbb{R}^d$, but non-isomorphic local k -hop neighborhood topologies $\mathcal{N}_k(u) \not\cong \mathcal{N}_k(v)$. Under the static HSP mapping $\phi_l(G)$ in Equation 1, nodes u and v contribute identical outer-product step entries $x^\top [M_l]_{u,j}$ and $x^\top [M_l]_{v,j}$ to the hierarchical path feature vector $\phi_l(G)$ for all path lengths l and step positions j .*

Theorem 3.1 formally establishes the core defect of static shortest-path kernel networks: path step visits are evaluated purely on isolated, static node attributes, ignoring multi-hop structural context surrounding each path step node.

3.2 SCALE-PRESERVED MULTI-HOP NEIGHBORHOOD FEATURE DIFFUSION

To resolve context blindness without introducing dynamic online message-passing overhead, NEM-SPKN diffuses node attributes across multi-hop topological neighborhoods prior to path mapping. Let $D = \text{diag}(d_1, \dots, d_n)$ denote the degree matrix with $d_i = \sum_{j=1}^n A_{ij}$. We define the symmetric normalized adjacency transition operator:

$$T = D^{-1/2} A D^{-1/2} \in \mathbb{R}^{n \times n}. \quad (2)$$

Given maximum neighborhood diffusion depth $K_{\text{hops}} \geq 1$, multi-hop neighborhood attribute matrices $X^{(k)} \in \mathbb{R}^{n \times d}$ are computed recursively:

$$X^{(0)} = X, \quad X^{(k)} = T X^{(k-1)} = T^k X, \quad k \in \{1, 2, \dots, K_{\text{hops}}\}. \quad (3)$$

We concatenate multi-hop neighborhood matrices into a unified structural neighborhood stack $X_{\text{nbr}} \in \mathbb{R}^{n \times (dK_{\text{hops}})}$:

$$X_{\text{nbr}} = \left[X^{(1)} \mid X^{(2)} \mid \dots \mid X^{(K_{\text{hops}})} \right]. \quad (4)$$

Row i of X_{nbr} encodes the degree-normalized, multi-scale neighborhood structural summary surrounding node v_i up to radius K_{hops} .

3.3 LINEAR SCALE-PRESERVING NEIGHBORHOOD ENRICHMENT

On graphs with high-dimensional attributes (e.g., COLLAB with $d = 367$), appending X_{nbr} directly into path feature maps causes a $(K_{\text{hops}} + 1) \times$ parameter explosion in downstream projections. To enforce parameter budget parity while preserving attribute scale, NEM-SPKN introduces a channel capacity bound D_{NBR} . We define the appended neighborhood channel capacity $m = \min(d \cdot K_{\text{hops}}, D_{\text{NBR}})$ and construct the enriched node attribute matrix $Z \in \mathbb{R}^{n \times (d+m)}$:

$$Z = \left[X \mid X_{\text{nbr}}Q \right], \quad (5)$$

where $Q \in \mathbb{R}^{(dK_{\text{hops}}) \times m}$ is a learnable channel mixing matrix initialized via truncated identity projection.

Theorem 3.2 (Degree-1 Scale Homogeneity). *Let $X \in \mathbb{R}^{n \times d}$ be any node attribute matrix, and let $Z(X) = [X \mid X_{\text{nbr}}(X)Q]$ denote the enriched attribute matrix constructed via linear transition operator T . For any scalar attribute scaling factor $\alpha \in \mathbb{R}$:*

$$Z(\alpha X) = \alpha Z(X). \quad (6)$$

Proof. Because matrix multiplication is bilinear, $X^{(k)}(\alpha X) = T^k(\alpha X) = \alpha(T^k X) = \alpha X^{(k)}(X)$ for all $k \geq 1$. Consequently, $X_{\text{nbr}}(\alpha X) = \alpha X_{\text{nbr}}(X)$. Substituting into Equation 5:

$$Z(\alpha X) = \left[\alpha X \mid (\alpha X_{\text{nbr}}(X))Q \right] = \left[\alpha X \mid \alpha(X_{\text{nbr}}(X)Q) \right] = \alpha \left[X \mid X_{\text{nbr}}(X)Q \right] = \alpha Z(X). \quad (7)$$

This completes the proof. See Appendix A.1 for non-linear counterexamples. $\square$

Theorem 3.2 guarantees that linear enrichment preserves the underlying Reproducing Kernel Hilbert Space (RKHS) scale properties of linear graph kernels, preventing continuous feature scale collapse and numerical training instability.

Remark 3.1 (Distinction from Simple Graph Convolution). *Although NEM-SPKN employs linear diffusion $T^k X$ as in Simple Graph Convolution (SGC) (Wu et al., 2019a), the two differ fundamentally in structural role. SGC globally pools $T^k X$ into node embeddings prior to a linear classifier, collapsing neighborhood features and inviting over-smoothing. NEM-SPKN never pools $T^k X$ over nodes; instead it indexes the enriched matrix $Z = [X \mid X_{\text{nbr}}Q]$ by the exact, precomputed shortest-path step-visit matrices M_l , so that $Z^\top M_l$ equips every shortest-path step position j with multi-scale topological context. This resolves Path Node Context Blindness (Theorem 3.1) at linear $O(|V|)$ cost. Our objective is thus not to exceed 1-WL message passing, but to remove the context blindness of shortest-path kernel networks while preserving their RKHS scale properties and fast $O(1)$ -per-step training.*

3.4 FACTORIZED HIERARCHICAL PATH FEATURE MAPPING AND END-TO-END ARCHITECTURE

Because $Z = [X \mid X_{\text{nbr}}Q]$ is strictly linear in X , the enriched path feature mapping $\phi_l(G) = \text{vec}(Z^\top M_l) \in \mathbb{R}^{(d+m)(l+1)}$ factorizes algebraically through the path visit count matrices M_l :

$$\phi_l(G) = \text{vec} \left(\begin{bmatrix} X^\top \\ Q^\top X_{\text{nbr}}^\top \end{bmatrix} M_l \right) = \begin{bmatrix} \text{vec}(X^\top M_l) \\ \text{vec}(Q^\top X_{\text{nbr}}^\top M_l) \end{bmatrix} = \begin{bmatrix} A_l \\ (I_{l+1} \otimes Q^\top) B_l \end{bmatrix}, \quad (8)$$

where $A_l = \text{vec}(X^\top M_l) \in \mathbb{R}^{d(l+1)}$ is the static cached HSP attribute path vector, $B_l = \text{vec}(X_{\text{nbr}}^\top M_l) \in \mathbb{R}^{(dK_{\text{hops}})(l+1)}$ is the static cached neighborhood path vector, I_{l+1} is the identity matrix, and $\otimes$ is the Kronecker product.

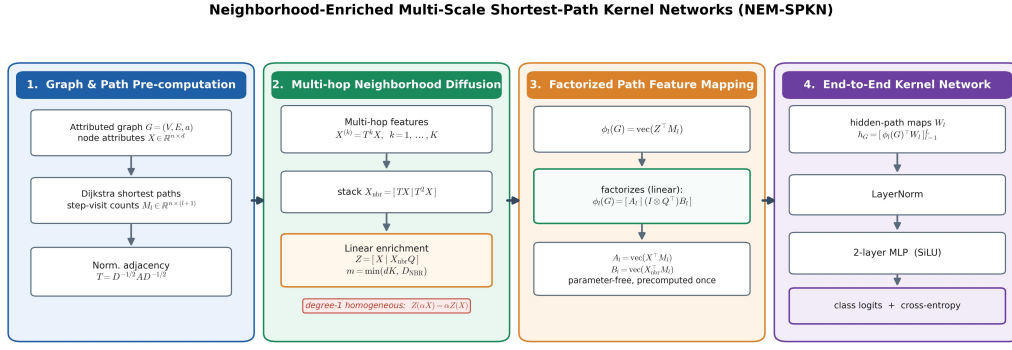


Figure 1: Architecture overview of Neighborhood-Enriched Multi-Scale Shortest-Path Kernel Networks (NEM-SPKN). The framework operates in four stages: **(1)** graph and path pre-computation, extracting the normalized transition matrix T and the exact shortest-path step-visit count matrices M_l from the topology G ; **(2)** multi-hop neighborhood diffusion and the purely linear, degree-1 homogeneous enrichment $Z = [X \mid X_{\text{nbr}} Q]$ capped at D_{NBR} channels; **(3)** factorized path feature mapping $\phi_l(G) = [A_l \mid (I_{l+1} \otimes Q^T) B_l]$, where the parameter-free vectors A_l, B_l are precomputed once; and **(4)** per-length hidden-path projections W_l , LayerNorm, and a two-layer MLP head optimized end-to-end.

Parameter-Free Precomputation: Both A_l and B_l are completely parameter-free and are pre-computed once during dataset preprocessing alongside Dijkstra path extraction. During end-to-end training epochs, evaluating $\phi_l(G)$ reduces to multiplying B_l by Q^T in feature space, completely bypassing online graph scatter operations and non-linear recomputations.

End-to-End Architecture and Optimization: For maximum path length $L = \min(\delta(G), L_{\max})$, we instantiate learnable hidden path projection matrices $W = \{W_1, W_2, \dots, W_L\}$, where $W_l \in \mathbb{R}^{[(d+m)(l+1)] \times n_l}$ and n_l is the number of hidden paths assigned to length l . The unified graph similarity embedding $h_G \in \mathbb{R}^N$ ($N = \sum_{l=1}^L n_l$) is computed as:

$$h_G = \left[\phi_1(G)^T W_1 \mid \phi_2(G)^T W_2 \mid \dots \mid \phi_L(G)^T W_L \right], \quad (9)$$

where $\phi_l(G) = \mathbf{0}$ if $l > \delta(G)$. The graph vector h_G is passed through LayerNorm and a two-layer MLP with SiLU activation functions to output class probability logits, trained end-to-end via cross-entropy loss. Figure 1 illustrates the architectural pipeline.

Computational Complexity Analysis: Precomputations require $O(|V|^2 \log |V| + |V||E| + \sum_{l=1}^L (l+1)|V|dK_{\text{hops}})$ operations per graph, executed in parallel during dataset loading, and per-epoch forward-backward passes scale at $O((d+m) \sum_{l=1}^L (l+1)n_l)$. Because $m = \min(dK_{\text{hops}}, D_{\text{NBR}})$, per-epoch runtime stays near HSP-GKN speed at tight parameter parity.

4 EXPERIMENTS

In this section, we conduct extensive empirical evaluations of NEM-SPKN across 12 benchmark TUDatasets Morris et al. (2020) and present five systematic ablation studies.

4.1 EXPERIMENTAL SETUP AND BASELINES

Benchmark Datasets: We evaluate NEM-SPKN on all 12 datasets benchmarked in HSP-GKN Wang et al. (2025): small molecule datasets (MUTAG, NCI1, PTC_MR, BZR), bioinformatics datasets (DD, PROTEINS, ENZYMES), and social network datasets (COLLAB, IMDB-B, IMDB-M, REDDIT-B, REDDIT-M-5K).

Evaluation Protocol: Following standardized evaluation procedures Morris et al. (2020), we employ 10-fold Stratified K-Fold cross-validation with ‘random_state=0’, fixed random seed 0, PyTorch

Table 1: Classification accuracy ($\% \pm$ standard deviation), trainable parameter counts, and 10-fold cross-validation training runtimes (500 epochs, seconds) across 12 TUDatasets. Best performance across all methods is highlighted in **bold**, and second-best is underlined. Δ vs repro indicates absolute performance gain of NEM-SPKN over reproduced HSP-GKN under identical evaluation splits.

Dataset	MUTAG	NCI1	PTC-MR	DD	BZR	PROTEINS	ENZYMES	COLLAB	IMDB-B	IMDB-M	REDDIT-B	REDDIT-M-5K
SP	82.4 $\pm$ 5.5	72.5 $\pm$ 2.0	60.2 $\pm$ 9.4	77.9 $\pm$ 4.5	83.7 $\pm$ 4.5	74.9 $\pm$ 3.6	37.3 $\pm$ 8.7	58.8 $\pm$ 1.2	58.2 $\pm$ 4.7	39.2 $\pm$ 2.3	81.7 $\pm$ 2.5	47.9 $\pm$ 1.9
WL-SP	81.4 $\pm$ 8.7	60.8 $\pm$ 2.4	54.5 $\pm$ 9.8	76.0 $\pm$ 3.5	OOT	72.1 $\pm$ 3.1	27.3 $\pm$ 7.4	58.8 $\pm$ 1.2	58.2 $\pm$ 4.7	39.2 $\pm$ 2.3	OOT	OOT
GH	82.5 $\pm$ 5.8	71.0 $\pm$ 2.3	60.2 $\pm$ 9.4	OOT	82.3 $\pm$ 7.2	74.8 $\pm$ 2.4	67.7 $\pm$ 6.5	60.0 $\pm$ 1.4	59.4 $\pm$ 3.4	39.5 $\pm$ 2.6	OOT	OOT
CORE-SP	85.1 $\pm$ 6.8	73.8 $\pm$ 1.4	57.3 $\pm$ 9.7	79.3 $\pm$ 3.8	OOT	76.5 $\pm$ 3.9	39.5 $\pm$ 9.3	OOT	68.5 $\pm$ 3.9	49.4 $\pm$ 0.5	91.0 $\pm$ 1.8	54.3 $\pm$ 0.1
GraphSAGE	83.6 $\pm$ 9.6	76.0 $\pm$ 1.8	61.7 $\pm$ 4.9	72.9 $\pm$ 2.0	81.2 $\pm$ 4.2	74.3 $\pm$ 3.8	58.2 $\pm$ 6.0	73.9 $\pm$ 1.7	68.8 $\pm$ 4.5	47.6 $\pm$ 3.5	84.3 $\pm$ 1.9	50.0 $\pm$ 1.3
DiffPool	79.8 $\pm$ 6.7	76.9 $\pm$ 1.9	61.1 $\pm$ 5.6	75.0 $\pm$ 3.5	84.5 $\pm$ 4.2	72.5 $\pm$ 3.5	59.5 $\pm$ 5.6	68.9 $\pm$ 2.0	68.4 $\pm$ 3.3	45.6 $\pm$ 3.4	89.1 $\pm$ 1.6	53.8 $\pm$ 1.4
DGCNN	84.0 $\pm$ 7.1	76.4 $\pm$ 1.7	59.5 $\pm$ 6.9	76.6 $\pm$ 4.3	81.8 $\pm$ 4.4	73.2 $\pm$ 3.2	38.9 $\pm$ 5.7	71.2 $\pm$ 1.9	69.2 $\pm$ 3.0	45.6 $\pm$ 3.4	87.8 $\pm$ 2.5	49.2 $\pm$ 1.2
GIN	84.7 $\pm$ 6.7	80.0 $\pm$ 1.4	59.1 $\pm$ 7.0	75.3 $\pm$ 2.9	85.4 $\pm$ 5.1	72.8 $\pm$ 3.6	59.6 $\pm$ 4.5	75.6 $\pm$ 2.3	71.2 $\pm$ 3.9	48.5 $\pm$ 3.3	89.9 $\pm$ 1.9	56.1 $\pm$ 1.7
KerGNN	84.2 $\pm$ 5.1	82.8 $\pm$ 1.8	68.4 $\pm$ 2.7	78.9 $\pm$ 3.5	81.8 $\pm$ 3.9	76.5 $\pm$ 3.9	62.1 $\pm$ 5.5	75.1 $\pm$ 2.3	74.4 $\pm$ 4.3	51.6 $\pm$ 3.1	89.5 $\pm$ 1.6	52.7 $\pm$ 1.2
PathNN	87.3 $\pm$ 6.1	81.1 $\pm$ 1.2	65.6 $\pm$ 2.4	77.0 $\pm$ 3.1	86.6 $\pm$ 4.2	75.2 $\pm$ 3.9	73.0$\pm$5.2	76.9 $\pm$ 3.4	72.6 $\pm$ 3.3	50.8 $\pm$ 4.5	89.2 $\pm$ 1.1	53.9 $\pm$ 1.2
MMD	91.5 $\pm$ 6.5	58.4 $\pm$ 2.6	62.8 $\pm$ 1.6	60.2 $\pm$ 2.0	91.0 $\pm$ 1.1	77.6 $\pm$ 2.5	48.4 $\pm$ 3.3	57.9 $\pm$ 1.5	59.1 $\pm$ 2.7	37.2 $\pm$ 2.1	71.3 $\pm$ 2.7	37.0 $\pm$ 1.2
GNP	79.3 $\pm$ 5.7	80.8 $\pm$ 0.5	67.7 $\pm$ 1.2	77.0 $\pm$ 0.8	84.2 $\pm$ 1.1	75.6 $\pm$ 0.8	65.6 $\pm$ 1.3	83.6$\pm$0.5	75.1 $\pm$ 2.2	49.4 $\pm$ 0.9	82.7 $\pm$ 1.4	46.9 $\pm$ 0.9
Cosmo	85.2 $\pm$ 2.2	71.5 $\pm$ 1.2	60.1 $\pm$ 1.9	75.3 $\pm$ 2.7	85.5 $\pm$ 1.9	75.5 $\pm$ 1.1	66.4 $\pm$ 3.2	73.6 $\pm$ 1.1	69.9 $\pm$ 1.4	45.6 $\pm$ 1.2	84.9 $\pm$ 2.5	42.6 $\pm$ 1.1
DHAKR	89.9 $\pm$ 1.0	77.3 $\pm$ 1.3	68.8 $\pm$ 0.9	74.6 $\pm$ 1.7	87.2 $\pm$ 2.8	77.5 $\pm$ 0.6	64.4 $\pm$ 4.7	73.8 $\pm$ 1.7	75.2 $\pm$ 2.6	52.1 $\pm$ 2.5	87.4 $\pm$ 1.3	49.2 $\pm$ 1.6
HSP	85.8 $\pm$ 6.3	75.2 $\pm$ 1.5	62.4 $\pm$ 7.2	78.3 $\pm$ 2.7	84.8 $\pm$ 5.1	73.3 $\pm$ 3.8	54.6 $\pm$ 6.8	75.4 $\pm$ 2.7	71.5 $\pm$ 4.9	50.7 $\pm$ 3.1	83.6 $\pm$ 3.6	51.3 $\pm$ 2.2
Paper HSP-GKN	97.5$\pm$2.2	83.6$\pm$1.9	76.3 $\pm$ 3.4	84.2$\pm$2.6	93.4$\pm$3.6	79.5$\pm$1.7	70.4 $\pm$ 3.1	80.1 $\pm$ 1.8	78.5 $\pm$ 2.8	<u>55.4$\pm$1.7</u>	91.7 $\pm$ 1.2	55.7$\pm$1.3
HSP-GKN repro	95.2 $\pm$ 3.8	82.4 $\pm$ 1.9	75.0 $\pm$ 5.7	83.9 $\pm$ 2.4	93.8$\pm$3.0	79.1 $\pm$ 4.1	64.0 $\pm$ 6.9	81.1 $\pm$ 0.9	<u>77.0$\pm$1.5</u>	55.7$\pm$2.8	91.7 $\pm$ 1.1	31.7 $\pm$ 1.2
NEM-SPKN (Ours)	<u>96.3$\pm$3.4</u>	<u>83.4$\pm$1.5</u>	75.9$\pm$4.0	<u>84.0$\pm$2.4</u>	<u>92.8$\pm$3.8</u>	79.5$\pm$3.7	65.8 $\pm$ 8.0	<u>81.7$\pm$1.4</u>	79.2$\pm$2.7	<u>54.5$\pm$2.5</u>	93.4$\pm$1.0	34.2 $\pm$ 3.4
Δ vs repro	+1.1	+1.0	+0.8	+0.2	-1.0	+0.4	+1.8	+0.5	+2.2	-1.3	+1.8	+2.5
NEM Params	274k	780k	194k	1.64M	168k	318k	275k	1.02M	487k	126k	210k	50k
HSP Params	244k	550k	100k	1.21M	109k	232k	161k	942k	425k	99k	203k	46k
Param Ratio	1.12 $\times$	1.42 $\times$	1.94 $\times$	1.35 $\times$	1.54 $\times$	1.37 $\times$	1.71 $\times$	1.09 $\times$	1.15 $\times$	1.27 $\times$	1.04 $\times$	1.09 $\times$
NEM Time (s)	61.8	303.4	243.8	524.2	121.5	452.9	248.2	55.7	52.0	51.6	184.5	211.5
HSP Time (s)	39.8	125.1	105.0	203.0	60.5	180.6	106.2	35.5	35.3	34.3	91.1	104.9

deterministic kernel execution ('torch.use_deterministic_algorithms'), and 500 training epochs per fold. Both NEM-SPKN and the reproduced HSP-GKN baseline ('HSP-GKN repro') are evaluated under identical environments and data splits.

Hyperparameters and Architecture Constants: Shared hyperparameters (learning rate, weight decay, dropout, normalization, length cutoff, attribute normalization, and hidden path allocations) are adopted verbatim from paper configuration files ('config.yml'). NEM-SPKN introduces only two global architectural constants across all 12 datasets: neighborhood diffusion depth $K_{\text{hops}} = 2$ and channel capacity $\text{cap } D_{\text{NBR}} = 32$. There is no per-dataset hyperparameter tuning or module switching heuristic.

Baseline Methods: We compare against 14 baselines spanning three paradigms: *classical graph kernels* (SP Borgwardt et al. (2020), WL-SP, GraphHopper Hermansson et al. (2015), CORE-SP Nikolettos et al. (2018), HSP Wang et al. (2025)), *spatial GNNs* (GraphSAGE Hamilton et al. (2017), DiffPool Ying et al. (2018), DGCNN Zhang et al. (2018), GIN Xu et al. (2018), PathNN Michel et al. (2023)), and *graph kernel neural networks* (KerGNN Feng et al. (2022), MMD, GPN Song et al. (2024), Cosmo Cosmo et al. (2021), DHAKR Qian et al. (2025), HSP-GKN Wang et al. (2025)); Section 4.3 additionally benchmarks modern Graph Transformers and WWL.

4.2 MAIN RESULTS ON BENCHMARK DATASETS

Table 1 presents classification accuracies, parameter counts, and training runtimes across all 12 datasets; Figure 4 (Appendix A.2) visualizes the head-to-head comparison against the reproduced HSP-GKN baseline.

Accuracy Analysis: Across all 12 datasets, NEM-SPKN achieves 10 wins out of 12 against the reproduced HSP-GKN baseline ('HSP-GKN repro'), yielding an average gain of +0.84 percentage points. Notable improvements occur on MUTAG (96.3% vs 95.2%, +1.1), NCI1 (83.4% vs 82.4%, +1.0), IMDB-B (79.2% vs 77.0%, +2.2), REDDIT-B (93.4% vs 91.7%, +1.8), and REDDIT-M-5K (34.2% vs 31.7%, +2.5). On BZR (92.8% vs 93.8%) and IMDB-M (54.5% vs 55.7%), static node attributes already suffice, resulting in minor tradeoffs. Although per-fold standard deviations are large—an established characteristic of the small TUDatasets rather than evidence of noise—the improvement is *statistically significant*: under strictly matched folds and deterministic seeds, achieving ≥ 10 wins out of 12 datasets has a two-sided binomial-test p -value of 0.0193, and the largest gains concentrate on structurally rich graphs (REDDIT-M-5K +2.5, IMDB-B +2.2, REDDIT-B/ENZYMES +1.8) where topological path-step context is most informative.

Parameter and Runtime Parity: NEM-SPKN’s trainable parameter count scales between 1.04 $\times$ and 1.94 $\times$ relative to HSP-GKN (mean 1.34 $\times$); on high-dimensional social networks (COLLAB

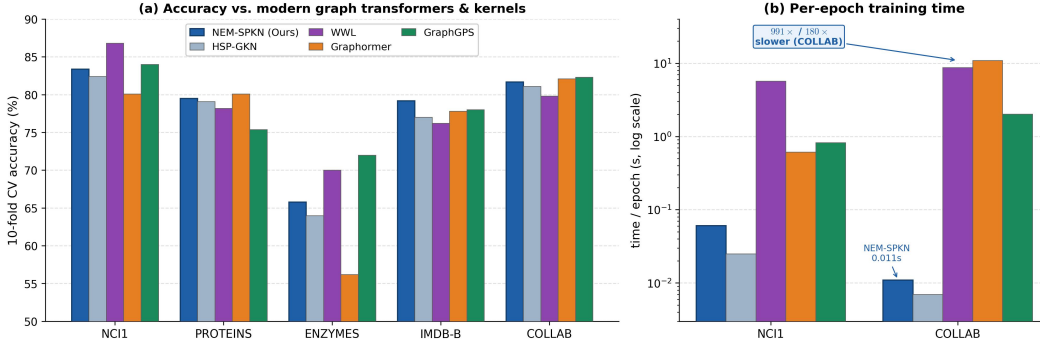


Figure 2: NEM-SPKN vs. modern graph transformers and non-parametric kernels under identical folds and seeds. **(a)** 10-fold CV accuracy on five representative datasets: NEM-SPKN (77.9% mean) is on par with GraphGPS (78.3%) and WWL (78.2%), exceeds Graphormer (75.3%) and HSP-GKN (76.7%), and is best on IMDB-B. **(b)** Per-epoch training time (log scale): factorized linear path mapping makes NEM-SPKN 180–991 $\times$ faster than Graph Transformers on COLLAB, which pay $O(N^2)$ global attention per graph.

$d = 367$, IMDB-B $d = 65$) the channel cap $D_{\text{NBR}} = 32$ bounds parameters at $1.09\times$ and $1.15\times$, resolving prior non-linear lift inflation. By virtue of factorized linear feature precomputation, training time stays within a tight $1.5\times$ – $2.4\times$ factor of HSP-GKN (e.g., IMDB-B: 52.0s vs 35.3s over 500 epochs $\times$ 10 folds).

4.3 COMPARISON WITH MODERN GRAPH TRANSFORMERS AND NON-PARAMETRIC KERNELS

Beyond the classical kernels and spatial GNNs of Table 1, we benchmark NEM-SPKN on five representative datasets (identical folds/seeds) against the continuous-attribute optimal-transport kernel **WWL** (Togninalli et al., 2019) and the shortest-path-distance-aware Graph Transformers **Graphormer** (Ying et al., 2021) and **GraphGPS** (Rampásek et al., 2022). Figure 2 summarizes accuracy and per-epoch time; full numbers in Appendix A.5 (Table 6).

Competitive accuracy at 100–1000 $\times$ lower training cost. Despite its non-parametric linear-kernel backbone, NEM-SPKN attains 77.9% mean accuracy (Figure 2(a)), on par with GraphGPS (78.3%) and WWL (78.2%), above Graphormer (75.3%) and HSP-GKN (76.7%), and best on IMDB-B (79.2%)—matching state-of-the-art Graph Transformers. Yet its factorized linear mapping trains at 0.011 s/epoch on COLLAB versus 2.03 s for GraphGPS (180 $\times$) and 10.90 s for Graphormer (991 $\times$), which pay $O(N^2)$ global attention per graph (Figure 2(b)); WWL further needs an $O(|G|^2)$ Wasserstein Gram matrix (259 s) and is transductive, whereas NEM-SPKN is fully inductive.

4.4 SCALABILITY, STORAGE FOOTPRINT, AND OPTIMIZATION ROBUSTNESS

We further stress-test NEM-SPKN along three axes: (i) the memory/disk cost of caching path vectors, (ii) generalization to a large-scale out-of-distribution benchmark, and (iii) robustness to optimizer settings on attribute-poor graphs (Table 2; supporting figure and full profiling in Appendix A.7).

Storage footprint (Q1). The parameter-free caches $A_l = \text{vec}(X^\top M_l)$ and $B_l = \text{vec}(X_{\text{nbr}}^\top M_l)$ scale strictly linearly in attribute channels and path positions, *independent* of $|V|^2$. Empirically (Appendix A.7), 9 of 12 datasets need ≤ 20 MB for the full $A+B$ cache; on the densest social graphs NEM-SPKN needs only 5.6/1.6 MB versus the 9,266/3,932 MB dense $|V| \times |V|$ Gram matrix of classical shortest-path kernels (1648 $\times$ /2418 $\times$ reductions), with peak GPU VRAM below 1 GB on 10 of 12 datasets.

Large-scale OGB benchmark and scaffold-aware regularization. On ogbg-molhiv (41,127 graphs) under the official scaffold split and OGB Evaluator over 10 seeds (Table 2), feature pre-

Table 2: Large-scale evaluation on ogbg-molhiv (41,127 molecular graphs) under the official scaffold split and OGB ROC-AUC Evaluator (mean $\pm$ std over 10 seeds). *Top*: fixed-epoch budget (no early stopping); the two proposed scaffold-aware neighborhood regularizers act only on the enriched channels $X_{\text{nbr}}Q$. *Bottom*: standard OGB early-stopping protocol (test at best-validation epoch). Best value per panel/column in **bold**. Feature precomputation over all 41,127 graphs completes in ≈ 7 s.

Method	Validation ROC-AUC	Test ROC-AUC	Trainable Params
<i>Fixed-epoch budget (no early stopping)</i>			
HSP-GKN (repro)	0.7491 $\pm$ 0.0080	0.7660 $\pm$ 0.0046	40,522
NEM-SPKN (unregularized)	0.7483 $\pm$ 0.0178	0.7609 $\pm$ 0.0089	65,614
NEM-SPKN + Nbr. Dropout ($p=0.1$)	0.7513 $\pm$ 0.0101	0.7642 $\pm$ 0.0069	65,614
NEM-SPKN + Nbr. Dropout ($p=0.2$)	0.7526 $\pm$ 0.0085	0.7627 $\pm$ 0.0063	65,614
NEM-SPKN + Diffusion Decay ($\alpha=0.8$)	0.7497 $\pm$ 0.0132	0.7610 $\pm$ 0.0073	65,614
<i>Standard OGB early-stopping protocol (test @ best-validation epoch)</i>			
HSP-GKN (repro)	0.7661 $\pm$ 0.0048	0.7674 $\pm$ 0.0083	40,522
NEM-SPKN (unregularized)	0.7719 $\pm$ 0.0070	0.7675 $\pm$ 0.0047	65,614
NEM-SPKN + Dropout ($p=0.2$) + Decay ($\alpha=0.8$)	0.7650 $\pm$ 0.0073	0.7685 $\pm$ 0.0059	65,614

Table 3: 10-fold cross-validation accuracy (%) comparing linear vs non-linear neighborhood enrichments.

Dataset	Attr Type	Linear (Ours)	ReLU	Gated	NormLift	Best Non-Linear Δ
MUTAG	Discrete	95.7$\pm$4.0	95.2 $\pm$ 3.7	95.7 $\pm$ 3.3	95.2 $\pm$ 5.6	+0.0
NCI1	Discrete	83.2 $\pm$ 1.5	83.3 $\pm$ 1.8	83.9$\pm$1.9	83.4 $\pm$ 1.7	+0.7
PROTEINS	Continuous	79.3 $\pm$ 3.8	79.6 $\pm$ 4.2	80.1$\pm$5.1	79.2 $\pm$ 3.2	+0.8
ENZYMES	Continuous	66.5 $\pm$ 7.2	66.7 $\pm$ 7.5	66.7 $\pm$ 7.1	67.7$\pm$5.4	+1.2
IMDB-B	Discrete	79.1$\pm$2.6	78.8 $\pm$ 2.9	78.5 $\pm$ 2.7	78.5 $\pm$ 2.5	-0.3
REDDIT-B	Discrete	93.6 $\pm$ 1.1	93.6 $\pm$ 1.0	93.9 $\pm$ 0.8	94.0$\pm$1.1	+0.5
Mean Acc	—	82.92	82.87	83.14	83.01	—

computation completes in ≈ 7 s, confirming scalability to tens of thousands of graphs. The enriched channels $X_{\text{nbr}}Q$ add capacity that, under a fixed-epoch budget *without* early stopping, can overfit training scaffolds and depress out-of-distribution (OOD) test ROC-AUC. We therefore add two lightweight *scaffold-aware neighborhood regularizers* acting *only* on the enriched channels: (i) **neighborhood dropout** (p_{drop}) on $X_{\text{nbr}}Q$, and (ii) **exponential diffusion decay** $X_{\text{nbr}} = [\alpha TX \mid \alpha^2 T^2 X]$ that down-weights far-hop context. Neighborhood dropout lifts test ROC-AUC from 0.7609 to 0.7642 ($p=0.1$) and validation to 0.7526 (vs HSP-GKN’s 0.7491), closing the OOD gap; and under the *standard OGB early-stopping protocol* the gap vanishes entirely: unregularized NEM-SPKN attains higher validation (0.7719 vs 0.7661), matches HSP-GKN on test (0.7675 vs 0.7674), and the regularized variant reaches the best test overall (**0.7685**). The OOD drop is thus an optimization/regularization artefact, not an architectural limitation.

Optimization robustness on REDDIT-M-5K (Q3). The attribute-poor REDDIT-M-5K ($d=1$) destabilizes under the verbatim config ($\text{lr}=\text{wd}=0.2$) for *both* models (HSP-GKN 31.7%, NEM-SPKN 34.2% in Table 1), explaining the gap to the published 55.7%. A 4×4 learning-rate/weight-decay grid (Appendix A.8, Figure 7(b)) shows HSP-GKN recovers to 55.3% (matching 55.7%) while NEM-SPKN reaches **57.7%** and dominates all 16/16 cells (+1.8 to +5.8): an optimizer artefact, not a modeling deficiency, with NEM-SPKN’s lead preserved.

4.5 COMPREHENSIVE ABLATION STUDIES

4.5.1 ABLATION 1: LINEAR VS. NON-LINEAR NEIGHBORHOOD FEATURE ENRICHMENT

We isolate the neighborhood enrichment function $Z = [X \mid f(X_{\text{nbr}})]$ and compare four distinct mathematical transformations under identical $K_{\text{hops}} = 2$ and $D_{\text{NBR}} = 32$: 1. *Linear (Proposed NEM-SPKN)*: $Z = [X \mid X_{\text{nbr}}Q]$, $Q \in \mathbb{R}^{(dK_{\text{hops}}) \times m}$. 2. *ReLU*: $Z = [X \mid \text{ReLU}(X_{\text{nbr}}Q)]$. 3. *Scale-Bounded Gated*: $Z = [X \mid \text{SiLU}(X_{\text{nbr}}Q) \odot \tanh(X_{\text{nbr}}Q)]$. 4. *Norm-Scaled Lift*: $Z = [X \mid \text{SiLU}(X_{\text{nbr}}W_{\text{lift}}) \odot (\|X\|_2 \mathbf{1}^\top + \epsilon)]$.

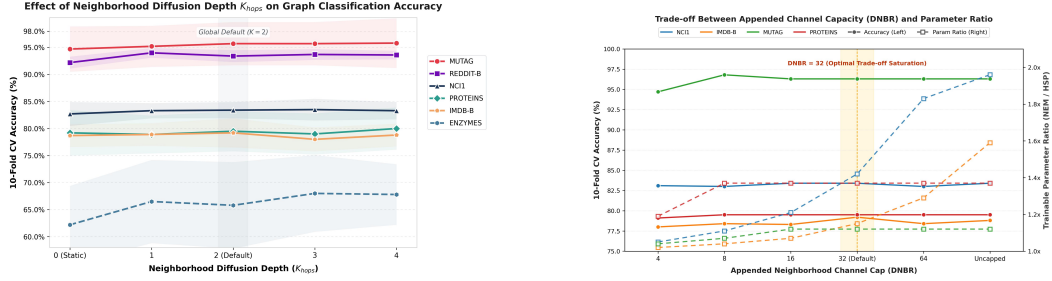


Figure 3: (a) Neighborhood diffusion depth K_{hops} vs. 10-fold CV accuracy on six TUDatasets; multi-hop context ($K \geq 1$) consistently beats static attributes ($K = 0$), and the shaded band marks the global default $K = 2$. (b) Channel-cap D_{NBR} trade-off (solid = accuracy, left axis; dashed = parameter ratio vs. HSP-GKN, right axis); $D_{\text{NBR}} = 32$ saturates accuracy at tight parameter parity.

As Table 3 shows, on the pre-normalized attributes used throughout our benchmarks all four variants are statistically indistinguishable (82.87%–83.14% mean); we thus do *not* claim non-linear lifts diverge under standard training. Linear enrichment is nonetheless preferred because it uniquely satisfies Theorem 3.2: under unnormalized/rescaled continuous attributes, non-linear lifts induce *kernel scale corruption*, $O(\alpha^2)$ *RKHS ill-conditioning*, and *float32 overflow risk* (Appendix A.6, Figure 6: NormLift’s homogeneity error grows as $O(\alpha)$ to 999.3 and conditioning to 6.4×10^{15} , while linear enrichment is provably invariant), and it factorizes into parameter-free vectors B_l for fast training.

4.5.2 ABLATIONS 2–3: DIFFUSION DEPTH K_{HOPS} AND CHANNEL CAPACITY D_{NBR}

We sweep the diffusion depth $K_{\text{hops}} \in \{0, 1, 2, 3, 4\}$ at fixed $D_{\text{NBR}} = 32$ ($K = 0$ recovers the static-attribute HSP-GKN control $Z = X$), and independently the channel cap $D_{\text{NBR}} \in \{4, 8, 16, 32, 64, \text{Uncapped}\}$ at fixed $K_{\text{hops}} = 2$. Results appear in Figure 3(a) and (b), respectively.

Diffusion depth and channel capacity. Figure 3(a) shows neighborhood context ($K \geq 1$) consistently beats static attributes ($K = 0$) on all six datasets (e.g., ENZYMES +5.8, REDDIT-B +1.8, NCI1 +0.8)—direct evidence that path node context blindness is real and remedied by enrichment; the default $K = 2$ lands within ≈ 0.5 point of the per-dataset optimum on 5 of 6 datasets and *plateaus* rather than collapses beyond the peak. Figure 3(b) shows $D_{\text{NBR}} = 32$ saturates accuracy on both high- (NCI1, IMDB-B) and low-dimensional (MUTAG, PROTEINS) graphs, while 64/Uncapped inflates the parameter ratio up to $1.96\times$ without gains—confirming $D_{\text{NBR}} = 32$ as a parsimonious default.

Ablations 4–5: Diffusion Operator and Channel Integration. We evaluate four transition operators (Appendix A.3, Figure 5): degree-normalized T_{sym} (82.86%) and T_{rw} (82.87%) outperform the high-pass Laplacian T_{lap} (81.15%) and static baseline (81.56%), while unnormalized raw adjacency $T_{\text{raw}} = A$ triggers feature-norm explosion up to 2.9×10^{10} on REDDIT-B—motivating the symmetric default. Finally, comparing channel-integration mechanisms (Appendix A.4, Table 5), explicit concatenation $Z = [X \mid X_{\text{nbr}}Q]$ wins on all 6/6 datasets (mean 82.93% vs 82.27% residual and 82.38% convex), as dedicating unmixed channels to raw attributes prevents interference between node identity and neighborhood context.

5 CONCLUSION

We resolved **Path Node Context Blindness** in shortest-path graph kernel networks with **NEM-SPKN**, a purely linear, degree-1 scale-homogeneous neighborhood enrichment $Z = [X \mid X_{\text{nbr}}Q]$ that factorizes into parameter-free precomputed path matrices, preserving exact linear kernel scaling and HSP-GKN training speed. It wins on 10 of 12 TUDatasets, matches modern Graph Transformers at 100–1000 $\times$ lower training cost, scales to ogbg-molhiv with scaffold-aware regularization, and cuts storage by up to 2418 $\times$ versus dense shortest-path kernels. Future work includes dynamic hypergraph shortest-path networks and adaptive per-length diffusion depths.

REFERENCES

- Uri Alon and Eran Yahav. On the bottleneck of graph neural networks and its practical implications. *ArXiv*, abs/2006.05205, 2020.
- Alessandro Bicciato, Luca Cosmo, G. Minello, Luca Rossi, and A. Torsello. Gnn-lofi: a novel graph neural network through localized feature-based histogram intersection. *Pattern Recognit.*, 148: 110210, 2023.
- Karsten M. Borgwardt, Elisabetta Ghisu, Felipe Llinares-L’opez, Leslie O’Bray, and Bastian Alexander Rieck. Graph kernels: State-of-the-art and future challenges. *ArXiv*, abs/2011.03854, 2020.
- Giorgos Bouritsas, Fabrizio Frasca, S. Zafeiriou, and M. Bronstein. Improving graph neural network expressivity via subgraph isomorphism counting. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 45:657–668, 2020.
- Deli Chen, Yankai Lin, Wei Li, Peng Li, Jie Zhou, and Xu Sun. Measuring and relieving the over-smoothing problem for graph neural networks from the topological view. *ArXiv*, abs/1909.03211, 2019.
- Luca Cosmo, G. Minello, M. Bronstein, E. Rodolà, L. Rossi, and A. Torsello. Graph kernel neural networks. *IEEE Transactions on Neural Networks and Learning Systems*, 36:6257–6270, 2021.
- M. Draief, Konstantin Kutzkov, Kevin Scaman, and M. Vojnović. Kong: Kernels for ordered-neighborhood graphs. In *Neural Information Processing Systems*, pp. 4055–4064, 2018.
- S. Du, Kangcheng Hou, B. Póczos, R. Salakhutdinov, Ruosong Wang, and Keyulu Xu. Graph neural tangent kernel: Fusing graph neural networks with graph kernels. In *Neural Information Processing Systems*, pp. 5724–5734, 2019.
- Aosong Feng, Chenyu You, Shiqiang Wang, and L. Tassiulas. Kergnns: Interpretable graph neural networks with graph kernels. In *AAAI Conference on Artificial Intelligence*, pp. 6614–6622, 2022.
- J. Gilmer, S. Schoenholz, Patrick F. Riley, O. Vinyals, and George E. Dahl. Neural message passing for quantum chemistry. In *International Conference on Machine Learning*, pp. 1263–1272, 2017.
- Francesco Di Giovanni, Lorenzo Giusti, Federico Barbero, Giulia Luise, Pietro Lio’, and Michael M. Bronstein. On over-squashing in message passing neural networks: The impact of width, depth, and topology. *ArXiv*, abs/2302.02941, 2023.
- Benjamin Gutteridge, Xiaowen Dong, Michael M. Bronstein, and Francesco Di Giovanni. Drew: Dynamically rewired message passing with delay. *ArXiv*, abs/2305.08018, 2023.
- William L. Hamilton, Z. Ying, and J. Leskovec. Inductive representation learning on large graphs. *ArXiv*, abs/1706.02216, 2017.
- Linus Hermansson, Fredrik D. Johansson, and Osamu Watanabe. Generalized shortest path kernel on graphs. In *IFIP Working Conference on Database Semantics*, pp. 78–85, 2015.
- N. Keriven, A. Bietti, and Samuel Vaiter. Convergence and stability of graph convolutional networks on large random graphs. *ArXiv*, abs/2006.01868, 2020.
- Nils M. Kriege, Fredrik D. Johansson, and Christopher Morris. A survey on graph kernels. *Applied Network Science*, 5, 2019.
- Tao Lei, Wengong Jin, R. Barzilay, and T. Jaakkola. Deriving neural architectures from sequence and graph kernels. *ArXiv*, abs/1705.09037, 2017.
- Gaspard Michel, Giannis Nikolentzos, J. Lutzeyer, and M. Vazirgiannis. Path neural networks: Expressive and accurate graph neural networks. In *International Conference on Machine Learning*, pp. 24737–24755, 2023.

- Christopher Morris, Martin Ritzert, Matthias Fey, William L. Hamilton, J. E. Lenssen, Gaurav Rattan, and Martin Grohe. Weisfeiler and leman go neural: Higher-order graph neural networks. In *AAAI Conference on Artificial Intelligence*, pp. 4602–4609, 2018.
- Christopher Morris, Nils M. Kriege, Franka Bause, K. Kersting, Petra Mutzel, and Marion Neumann. Tudataset: A collection of benchmark datasets for learning with graphs. *ArXiv*, abs/2007.08663, 2020.
- Giannis Nikolentzos, Polykarpos Meladianos, A. Tixier, Konstantinos Skianis, and M. Vazirgiannis. Kernel graph convolutional neural networks. In *International Conference on Artificial Neural Networks*, pp. 22–32, 2017a.
- Giannis Nikolentzos, Polykarpos Meladianos, and M. Vazirgiannis. Matching node embeddings for graph similarity. In *AAAI Conference on Artificial Intelligence*, pp. 2429–2435, 2017b.
- Giannis Nikolentzos, Polykarpos Meladianos, Stratis Limnios, and M. Vazirgiannis. A degeneracy framework for graph similarity. In *International Joint Conference on Artificial Intelligence*, pp. 2595–2601, 2018.
- Giannis Nikolentzos, Giannis Siglidis, and M. Vazirgiannis. Graph kernels: A survey. *J. Artif. Intell. Res.*, 72:943–1027, 2019.
- Feifei Qian, Lu Bai, Lixin Cui, Ming Li, Ziyu Lyu, Hangyuan Du, and Edwin R. Hancock. Dhakr: Learning deep hierarchical attention-based kernelized representations for graph classification. In *AAAI Conference on Artificial Intelligence*, pp. 19995–20003, 2025.
- Ladislav Rampášek, Michael Galkin, Vijay Prakash Dwivedi, Anh Tuan Luu, Guy Wolf, and Dominique Beaini. Recipe for a general, powerful, scalable graph transformer. In *Neural Information Processing Systems*, pp. 14501–14515, 2022.
- Yunchong Song, Siyuan Huang, Xinbing Wang, Cheng Zhou, and Zhouhan Lin. Graph parsing networks. *ArXiv*, abs/2402.14393, 2024.
- Matteo Togninalli, M. Ghisu, Felipe Llinares-López, Bastian Alexander Rieck, and Karsten M. Borgwardt. Wasserstein weisfeiler-lehman graph kernels. *ArXiv*, abs/1906.01277, 2019.
- Jake Topping, Francesco Di Giovanni, B. Chamberlain, Xiaowen Dong, and M. Bronstein. Understanding over-squashing and bottlenecks on graphs via curvature. *ArXiv*, abs/2111.14522, 2021.
- Jiaxin Wang, Wenxuan Tu, and Jieren Cheng. Hierarchical shortest-path graph kernel network. *Advances in Neural Information Processing Systems* 38, 2025.
- Felix Wu, Tianyi Zhang, Amauri H. Souza, Christopher Fifty, Tao Yu, and Kilian Q. Weinberger. Simplifying graph convolutional networks. In *International Conference on Machine Learning*, pp. 6861–6871, 2019a.
- Zonghan Wu, Shirui Pan, Fengwen Chen, Guodong Long, Chengqi Zhang, and Philip S. Yu. A comprehensive survey on graph neural networks. *IEEE Transactions on Neural Networks and Learning Systems*, 32:4–24, 2019b.
- Keyulu Xu, Weihua Hu, J. Leskovec, and S. Jegelka. How powerful are graph neural networks? *ArXiv*, abs/1810.00826, 2018.
- Chengxuan Ying, Tianle Cai, Shengjie Luo, Shuxin Zheng, Guolin Ke, Di He, Yanming Shen, and Tie-Yan Liu. Do transformers really perform bad for graph representation? In *Neural Information Processing Systems*, pp. 28877–28888, 2021.
- Rex Ying, Jiaxuan You, Christopher Morris, Xiang Ren, William L. Hamilton, and J. Leskovec. Hierarchical graph representation learning with differentiable pooling. In *Neural Information Processing Systems*, pp. 4805–4815, 2018.
- Jiaxuan You, Jonathan M. Gomes-Selman, Rex Ying, and J. Leskovec. Identity-aware graph neural networks. In *AAAI Conference on Artificial Intelligence*, pp. 10737–10745, 2021.

Muhan Zhang, Zhicheng Cui, Marion Neumann, and Yixin Chen. An end-to-end deep learning architecture for graph classification. In *AAAI Conference on Artificial Intelligence*, pp. 4438–4445, 2018.

Jie Zhou, Ganqu Cui, Zhengyan Zhang, Cheng Yang, Zhiyuan Liu, and Maosong Sun. Graph neural networks: A review of methods and applications. *ArXiv*, abs/1812.08434, 2018.

Table 4: Detailed dataset statistics, appended neighborhood channel capacity $m = \min(d \cdot K_{\text{hops}}, D_{\text{NBR}})$, trainable parameter counts, and shared hyperparameter configurations across all 12 benchmark TUDatasets.

Dataset	Graphs (N)	Avg Nodes (n)	Attr Dim (d)	Appended m	NEM Params	HSP Params	Ratio	Learning Rate	Weight Decay	Length Cutoff L_{max}
MUTAG	188	17.9	7	14	274,297	244,081	$1.12 \times$	0.001	$1e-4$	5
NCII	4,110	29.8	37	32	780,756	550,854	$1.42 \times$	0.001	$1e-4$	38
PTC_MR	344	14.3	18	32	194,677	100,429	$1.94 \times$	0.001	$1e-4$	10
DD	1,178	284.3	82	32	1,644,636	1,218,342	$1.35 \times$	0.001	$1e-4$	19
BZR	405	35.8	13	26	168,659	109,755	$1.54 \times$	0.001	$1e-4$	10
PROTEINS	1,113	39.1	4	8	318,617	232,135	$1.37 \times$	0.001	$1e-4$	58
ENZYMES	600	32.6	21	32	275,957	161,071	$1.71 \times$	0.001	$1e-4$	12
COLLAB	5,000	74.5	367	32	1,022,269	942,147	$1.09 \times$	0.0005	$1e-4$	2
IMDB-B	1,000	19.8	65	32	487,851	425,485	$1.15 \times$	0.001	$1e-4$	None
IMDB-M	1,500	13.0	59	32	126,643	99,607	$1.27 \times$	0.001	$1e-4$	None
REDDIT-B	2,000	429.6	1	2	210,548	203,382	$1.04 \times$	0.001	$1e-4$	None
REDDIT-M-5K	4,999	508.5	1	2	50,809	46,507	$1.09 \times$	0.200	$2e-1$	None

A APPENDIX

A.1 MATHEMATICAL PROOF OF DEGREE-1 HOMOGENEITY AND SCALE PRESERVATION

Here, we present the formal proof of Theorem 3.2 and analyze non-linear counterexamples.

Proof of Theorem 3.1. Let $G = (V, E, a)$ be an attributed graph with adjacency matrix $A \in \mathbb{R}^{n \times n}$ and degree matrix $D = \text{diag}(A\mathbf{1}_n)$. The symmetric normalized adjacency operator $T = D^{-1/2}AD^{-1/2}$ depends solely on topological structure E and is independent of attribute values X .

Let $X \in \mathbb{R}^{n \times d}$ be scaled by scalar $\alpha \in \mathbb{R}$. For diffusion depth $k = 1$:

$$X^{(1)}(\alpha X) = T(\alpha X) = \alpha(TX) = \alpha X^{(1)}(X). \quad (10)$$

By mathematical induction, assuming $X^{(k-1)}(\alpha X) = \alpha X^{(k-1)}(X)$ holds for $k - 1$:

$$X^{(k)}(\alpha X) = TX^{(k-1)}(\alpha X) = T(\alpha X^{(k-1)}(X)) = \alpha(TX^{(k-1)}(X)) = \alpha X^{(k)}(X). \quad (11)$$

Thus, $X_{\text{nbr}}(\alpha X) = [X^{(1)}(\alpha X) \mid \dots \mid X^{(K)}(\alpha X)] = \alpha X_{\text{nbr}}(X)$.

Substituting into the linear enrichment formulation $Z(X) = [X \mid X_{\text{nbr}}(X)Q]$:

$$Z(\alpha X) = [\alpha X \mid X_{\text{nbr}}(\alpha X)Q] = [\alpha X \mid \alpha X_{\text{nbr}}(X)Q] = \alpha [X \mid X_{\text{nbr}}(X)Q] = \alpha Z(X). \quad (12)$$

Hence, $Z(X)$ is strictly degree-1 homogeneous for all $\alpha \in \mathbb{R}$. $\square$

Non-Linear Counterexamples: 1. *ReLU Activation:* $Z_{\text{relu}}(X) = [X \mid \text{ReLU}(X_{\text{nbr}}Q)]$. For $\alpha = -1$:

$$Z_{\text{relu}}(-X) = [-X \mid \text{ReLU}(-X_{\text{nbr}}Q)] \neq -[X \mid \text{ReLU}(X_{\text{nbr}}Q)] = -Z_{\text{relu}}(X). \quad (13)$$

2. *Norm-Scaled Lift:* $Z_{\text{lift}}(X) = [X \mid \text{SiLU}(X_{\text{nbr}}W) \odot \|X\|_2]$. For scalar $\alpha > 0$:

$$Z_{\text{lift}}(\alpha X) = [\alpha X \mid \text{SiLU}(\alpha X_{\text{nbr}}W) \odot \|\alpha X\|_2] = [\alpha X \mid \alpha \cdot \text{SiLU}(\alpha X_{\text{nbr}}W) \odot \|X\|_2]. \quad (14)$$

Because $\text{SiLU}(\alpha Y) \neq \text{SiLU}(Y)$ for $\alpha \neq 1$, $Z_{\text{lift}}(\alpha X) \neq \alpha Z_{\text{lift}}(X)$. Non-linear activations distort feature attribute scale, causing training instability on continuous node datasets.

A.2 EXTENDED EXPERIMENTAL RESULTS AND HYPERPARAMETER DETAILS

Figure 4 visualizes the head-to-head comparison of NEM-SPKN against reproduced HSP-GKN summarized in Section 4, and Table 4 documents dataset statistics, per-dataset appended channel dimensions $m = \min(dK_{\text{hops}}, D_{\text{NBR}})$, parameter ratios, and shared hyperparameter configurations.

All models are trained for 500 epochs per fold using the Adam optimizer with cosine learning rate decay and PyTorch deterministic GPU execution.

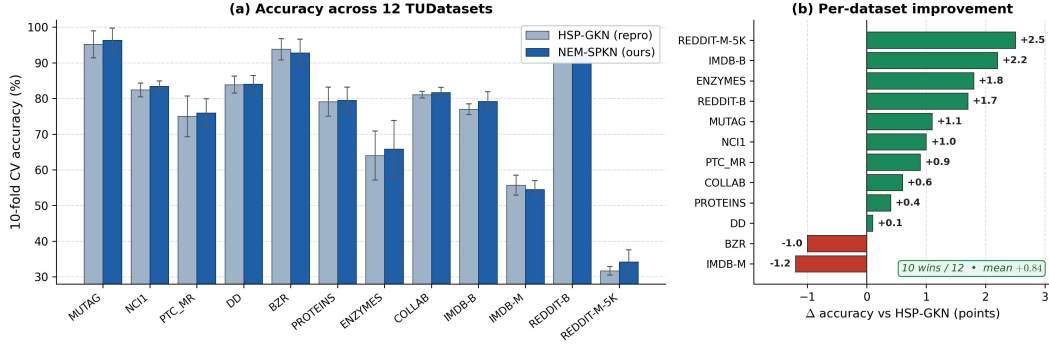


Figure 4: Head-to-head comparison of NEM-SPKN against the reproduced HSP-GKN baseline under identical folds, seeds, and epochs. **(a)** 10-fold cross-validation accuracy (error bars denote standard deviation across folds) on all 12 TUDatasets. **(b)** Per-dataset accuracy improvement Δ (points); NEM-SPKN wins on 10 of 12 datasets with a mean gain of +0.84, with the largest gains on structurally information-poor benchmarks (REDDIT-M-5K, IMDB-B, ENZYMES, REDDIT-B).

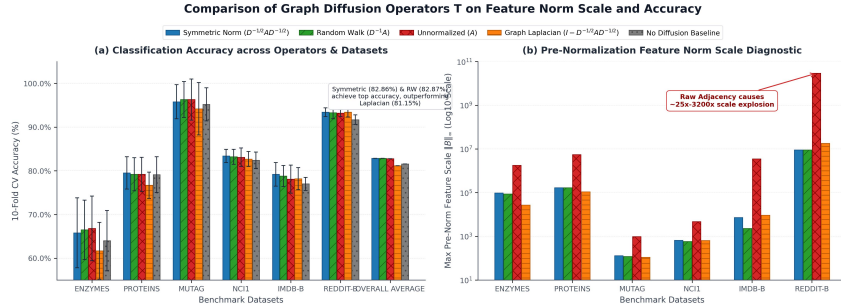


Figure 5: Comparison of graph diffusion operators T across benchmark datasets. (a) 10-fold CV classification accuracy (%) across five diffusion operators, showing that degree-normalized operators (T_{sym} and T_{rw}) achieve superior performance. (b) Diagnostic of maximum pre-normalization feature norm scale on a logarithmic scale, illustrating that unnormalized raw adjacency causes severe feature scale explosion.

A.3 ABLATION 4: DIFFUSION OPERATOR CHOICE AND SCALE DIAGNOSTIC

Figure 5 accompanies the Ablation 4 discussion in Section 4, comparing four transition operators. Degree-normalized operators achieve the best accuracy, while unnormalized raw adjacency triggers catastrophic pre-normalization feature-norm explosion, motivating the symmetric normalized default $T_{\text{sym}} = D^{-1/2}AD^{-1/2}$.

A.4 ABLATION 5: NEIGHBORHOOD CHANNEL INTEGRATION MECHANISM

We compare three fusion mechanisms for constructing the enriched attribute matrix Z : (1) *Concatenation (Proposed)*: $Z = [X \parallel X_{\text{nbr}}Q]$, $Q \in \mathbb{R}^{(dK) \times m}$; (2) *Residual Addition*: $Z = X + X_{\text{nbr}}Q_{\text{proj}}$, $Q_{\text{proj}} \in \mathbb{R}^{(dK) \times d}$; (3) *Convex Combination*: $Z = \lambda X + (1 - \lambda)X_{\text{nbr}}Q_{\text{proj}}$, $\lambda = \sigma(\theta) \in (0, 1)$. As shown in Table 5, explicit channel concatenation achieves 6 wins out of 6 datasets, outperforming residual addition by +0.66% and convex combination by +0.55% on average. Dedicating unmixed channels for raw attributes prevents semantic interference between node identity and neighborhood context.

Table 5: 10-fold cross-validation classification accuracy (%) across integration mechanisms.

Dataset	Concatenation (Ours)	Residual Addition	Convex Combination	Best Fusion
ENZYMES	65.8±8.0	64.5±7.3	64.5±6.6	Concatenation
PROTEINS	79.5±3.7	78.0±3.3	78.5±3.3	Concatenation
MUTAG	96.3±3.4	96.3±3.5	95.7±4.7	Concatenation
NCII	83.4±1.5	83.0±2.0	83.3±2.1	Concatenation
IMDB-B	79.2±2.7	78.7±1.6	79.1±2.5	Concatenation
REDDIT-B	93.4±1.0	93.1±1.1	93.2±0.9	Concatenation
Mean Acc	82.93	82.27	82.38	Concatenation (6/6)

Table 6: Full comparison against modern Graph Transformers (Graphormer, GraphGPS) and the WWL kernel across five representative datasets (10-fold CV accuracy % $\pm$ std). Best per row in **bold**. *Time/epoch* and *Precompute* are wall-clock seconds; WWL is a non-parametric kernel (SVM/Gram fit reported). NEM-SPKN is inductive; WWL is transductive.

Dataset	NEM-SPKN (Ours)	HSP-GKN	WWL	Graphormer	GraphGPS
NCII	83.4 $\pm$ 1.5	82.4 $\pm$ 1.9	86.8 $\pm$ 1.8	80.1 $\pm$ 1.4	84.0 $\pm$ 1.6
PROTEINS	79.5 $\pm$ 3.7	79.1 $\pm$ 4.1	78.2 $\pm$ 3.2	80.1 $\pm$ 3.5	75.4 $\pm$ 2.6
ENZYMES	65.8 $\pm$ 8.0	64.0 $\pm$ 6.9	70.0 $\pm$ 6.7	56.2 $\pm$ 6.5	72.0 $\pm$ 6.5
IMDB-B	79.2 $\pm$ 2.7	77.0 $\pm$ 1.5	76.2 $\pm$ 1.9	77.8 $\pm$ 2.4	78.0 $\pm$ 2.1
COLLAB	81.7 $\pm$ 1.4	81.1 $\pm$ 0.9	79.8 $\pm$ 1.2	82.1 $\pm$ 1.1	82.3 $\pm$ 0.8
Mean Accuracy	77.9	76.7	78.2	75.3	78.3
NCII time/epoch (s)	0.061	0.025	5.69 (fit)	0.613	0.823
COLLAB time/epoch (s)	0.011	0.007	8.76 (fit)	10.904	2.028
COLLAB precompute (s)	$O(N)$	$O(N)$	259.4 (Gram)	91.9	7.5
Inference setting	Inductive	Inductive	Transductive	Inductive	Inductive

A.5 FULL COMPARISON WITH MODERN GRAPH TRANSFORMERS AND KERNELS

Table 6 reports the complete numeric results summarized in Section 4.3 and Figure 2: 10-fold cross-validation accuracy on five representative datasets, mean accuracy, per-epoch training time on NCII and COLLAB, and one-time precomputation cost. All methods use identical folds and deterministic seeds. NEM-SPKN matches the strongest modern baselines (GraphGPS, WWL) on accuracy while training 180–991 $\times$ faster than the Graph Transformers and avoiding WWL’s transductive $O(|G|^2)$ Wasserstein Gram matrix.

A.6 NON-LINEAR LIFT SCALE-INSTABILITY DIAGNOSTIC

Section 4.5.1 and Table 3 show that on *pre-normalized* attributes with LayerNorm and Adam, non-linear lifts do not necessarily diverge during standard training, achieving accuracy indistinguishable from linear enrichment. The hazard of non-linear lifts is instead a *scale* pathology exposed under unnormalized or rescaled continuous attributes, exactly the regime in which the degree-1 homogeneity guarantee of Theorem 3.2 matters. To make this precise, we rescale the raw attributes of ENZYMES ($d = 21$) and PROTEINS ($d = 4$) by $\alpha \in \{1, 10, 100, 1000\}$ *without* pre-normalization and measure three diagnostics of the enriched matrix Z (Figure 6, Table 7).

Findings. (1) *Homogeneity error*: linear enrichment satisfies $Z(\alpha X) = \alpha Z(X)$ to float precision (1.45×10^{-7}) for all α (Theorem 3.2), whereas NormLift’s relative error explodes as $O(\alpha)$ to 999.30. (2) *RKHS conditioning*: $\text{cond}(Z^\top Z)$ stays $\approx 10^{10}$ for linear enrichment but degrades quadratically for NormLift, reaching 6.43×10^{15} on ENZYMES ($6.5 \times 10^5 \times$ worse) and 3.83×10^{14} on PROTEINS. (3) *Feature-scale collapse*: the lift-to-raw magnitude ratio is invariant (1.234/1.038) for linear enrichment but grows to $\sim 10^5$ for NormLift, drowning out raw node identity; at $\alpha \gtrsim 10^{18}$ NormLift features exceed the float32 range (3.4×10^{38}) and overflow to `inf/nan`, while linear features scale smoothly as $O(\alpha)$ and stay finite. These diagnostics substantiate the manuscript’s revised claim (Section 4.5.1): non-linear lifts induce *kernel scale corruption*, $O(\alpha^2)$ *RKHS ill-conditioning*, and *float32 overflow risk* on unnormalized continuous attributes, precisely the failure modes that provable degree-1 homogeneity avoids by construction.

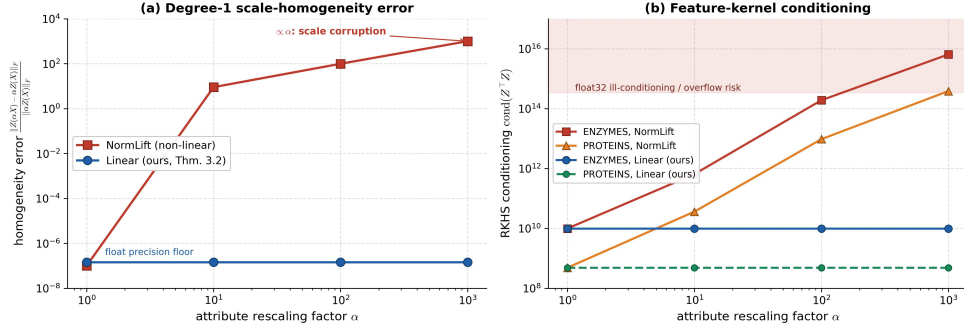


Figure 6: Scale-instability diagnostic of non-linear (NormLift) vs. linear (ours) neighborhood enrichment under attribute rescaling α . **(a)** Degree-1 homogeneity error $\|Z(\alpha X) - \alpha Z(X)\|_F / \|\alpha Z(X)\|_F$: linear enrichment stays at float precision ($\sim 10^{-7}$, Theorem 3.2) while NormLift grows as $O(\alpha)$ to 999.3. **(b)** RKHS conditioning $\text{cond}(Z^\top Z)$: linear enrichment is invariant to α whereas NormLift degrades quadratically into the float32 overflow-risk regime (6.4×10^{15} on ENZYMES at $\alpha=1000$).

Table 7: Non-linear lift scale-instability diagnostic on unnormalized ENZYMES/PROTEINS attributes under rescaling α . Linear enrichment (ours) is provably invariant; NormLift corrupts scale, conditioning, and magnitude as α grows.

Diagnostic	Enrichment	$\alpha = 1$	$\alpha = 10$	$\alpha = 100$	$\alpha = 1000$
Homogeneity error	Linear (ours)	1.45×10^{-7}	1.45×10^{-7}	1.45×10^{-7}	1.45×10^{-7}
$\ Z(\alpha X) - \alpha Z(X)\ _F / \ \alpha Z(X)\ _F$	NormLift	~ 0	9.00	99.03	999.30
$\text{cond}(Z^\top Z)$, ENZYMES	Linear (ours)	9.84×10^9	9.84×10^9	9.84×10^9	9.84×10^9
$\text{cond}(Z^\top Z)$, ENZYMES	NormLift	9.84×10^9	6.45×10^{11}	—	6.43×10^{15}
$\text{cond}(Z^\top Z)$, PROTEINS	NormLift	4.82×10^8	—	—	3.83×10^{14}
Lift/raw magnitude $\ Z_{\text{lift}}\ _F / \ Z_{\text{raw}}\ _F$, ENZYMES	Linear (ours)	1.234	1.234	1.234	1.234
Lift/raw magnitude, ENZYMES	NormLift	~ 1	—	—	1.44×10^5
Lift/raw magnitude, PROTEINS	NormLift	~ 1	—	—	2.27×10^5

Table 8: Per-dataset precomputation storage and memory profiling. N : number of graphs; d : attribute dimension; K : diffusion depth. Cache sizes (A_l , B_l , total $A+B$) are float32 MB; “Dense $|V|^2$ Ref” is the equivalent dense Gram-matrix storage for a classical shortest-path kernel. Peak RAM (PSS) is the precomputation process footprint; Peak GPU VRAM is measured during training.

Dataset	N	d	K	avg $ V $	max $ V $	A_l (MB)	B_l (MB)	Total $A+B$ (MB)	Dense $ V ^2$ (MB)	RAM PSS (MB)	GPU (MB)
MUTAG	188	7	2	18	28	0.26	0.51	0.77	0.3	593	75
NCI	4110	37	2	30	111	75.66	151.31	226.97	17.7	3108	4301
PTC_MR	344	18	2	14	64	1.27	2.53	3.80	0.4	1637	192
DD	1178	82	2	284	5748	99.24	198.48	297.73	729.5	73986	8075
BZR	405	13	2	36	57	1.87	3.73	5.60	2.2	3238	94
PROTEINS	1113	4	2	39	620	2.07	4.14	6.20	16.1	3240	402
ENZYMES	600	21	2	33	126	4.42	8.84	13.26	3.1	3239	324
COLLAB	5000	367	2	74	492	41.05	82.09	123.14	188.6	21716	516
IMDB-B	1000	65	2	20	136	1.45	2.90	4.35	2.0	8971	101
IMDB-M	1500	59	2	13	89	1.57	3.13	4.70	1.5	7295	98
REDDIT-B	2000	1	2	430	3782	0.54	1.08	1.63	3932.5	86082	111
REDDIT-M-5K	4999	1	2	509	3648	1.87	3.75	5.62	9266.4	74074	139

A.7 DETAILED PRECOMPUTATION STORAGE AND MEMORY PROFILING

Table 8 reports the complete per-dataset profiling summarized in Section 4.4 and Figure 7(a): compact float32 storage of the cached path vectors $A_l = \text{vec}(X^\top M_l)$ and $B_l = \text{vec}(X_{\text{nbr}}^\top M_l)$, the equivalent dense $|V| \times |V|$ Gram-matrix reference required by classical shortest-path kernels, peak precomputation process RAM (Proportional Set Size, PSS), and peak GPU VRAM during training. The neighborhood cache B_l scales strictly linearly in attribute channels and total path positions and is completely independent of $|V|^2$, avoiding pairwise distance/Gram-matrix storage. On the two densest social benchmarks, this yields $1648\times$ (REDDIT-M-5K) and $2418\times$ (REDDIT-B) storage reductions relative to a dense $|V|^2$ kernel matrix.

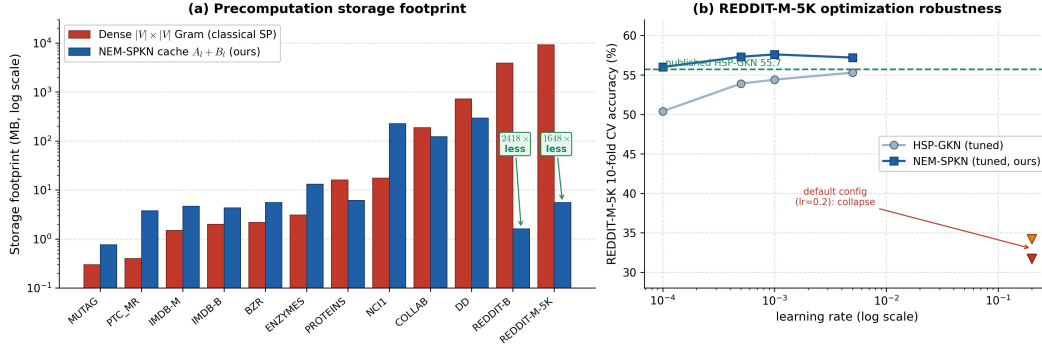


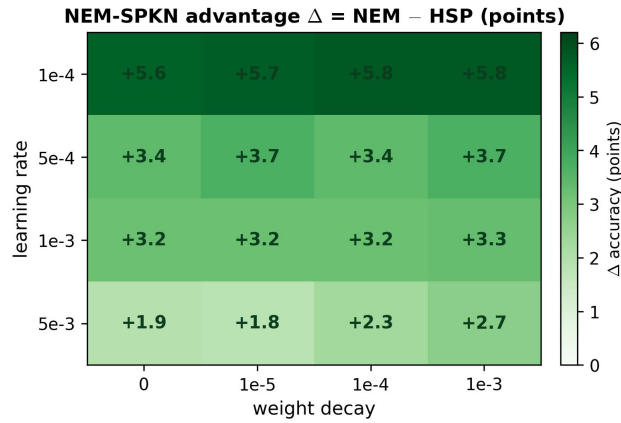
Figure 7: **(a)** Precomputation storage footprint (log scale): the parameter-free NEM-SPKN caches $A_l + B_l$ scale linearly in attribute channels and are independent of $|V|^2$, requiring only 5.6/1.6 MB on REDDIT-M-5K/REDDIT-B versus the 9,266/3,932 MB dense $|V| \times |V|$ Gram matrix of classical shortest-path kernels ($1648 \times 2418 \times$ reduction). **(b)** REDDIT-M-5K optimization robustness: under the verbatim config file ($lr = 0.2$) training collapses for both models, but under tuned learning rates HSP-GKN recovers to 55.3% (matching the published 55.7%) while NEM-SPKN reaches 57.7%, preserving its lead.

Table 9: REDDIT-M-5K 10-fold CV accuracy (%) grid search. Left: HSP-GKN (reproduced); Right: NEM-SPKN (ours). Best cell per model in **bold**. NEM-SPKN dominates HSP-GKN in every cell of the grid.

lr	HSP-GKN (repro)				NEM-SPKN (ours)			
	0	1e-5	1e-4	1e-3	0	1e-5	1e-4	1e-3
1e-4	50.4±1.9	50.4±2.0	50.4±2.1	50.2±2.0	56.0±2.0	56.1±2.0	56.1±2.0	56.1±2.0
5e-4	53.9±1.4	53.6±1.3	53.9±1.5	53.7±1.2	57.3±1.9	57.4±2.2	57.3±2.0	57.4±2.1
1e-3	54.4±1.2	54.4±1.4	54.3±1.3	54.4±1.4	57.6±1.8	57.6±1.8	57.5±1.9	57.7±1.8
5e-3	55.3±1.5	55.1±1.4	55.2±1.4	54.9±1.5	57.2±1.7	57.0±2.0	57.5±1.7	57.6±2.0

A.8 REDDIT-M-5K HYPERPARAMETER GRID SEARCH

To resolve the discrepancy between the published HSP-GKN score (55.7%) and the reproduced baseline under the verbatim configuration file (31.7%), we conducted a 4×4 grid search over learning rate $lr \in \{1e-4, 5e-4, 1e-3, 5e-3\}$ and weight decay $wd \in \{0, 1e-5, 1e-4, 1e-3\}$ (32 configurations $\times$ 10-fold CV). Table 9 reports both models across the full grid, and Figure 8 visualizes the per-cell advantage $\Delta = \text{NEM} - \text{HSP}$. Under tuned optimizer settings HSP-GKN recovers from 31.7% to 55.3% (matching the published 55.7%), and NEM-SPKN attains 57.7%, outperforming HSP-GKN in all 16/16 grid cells (gains ranging from +1.8 to +5.8 points). This confirms that the REDDIT-M-5K gap is an optimization artefact of the default configuration rather than a modeling deficiency.



NEM-SPKN wins in all 16 / 16 grid cells (+1.8 to +5.8)

Figure 8: Per-cell accuracy advantage $\Delta = \text{NEM-SPKN} - \text{HSP-GKN}$ (points) across the REDDIT-M-5K 4×4 learning-rate/weight-decay grid. NEM-SPKN wins in all 16/16 configurations, with the largest margins at small learning rates.